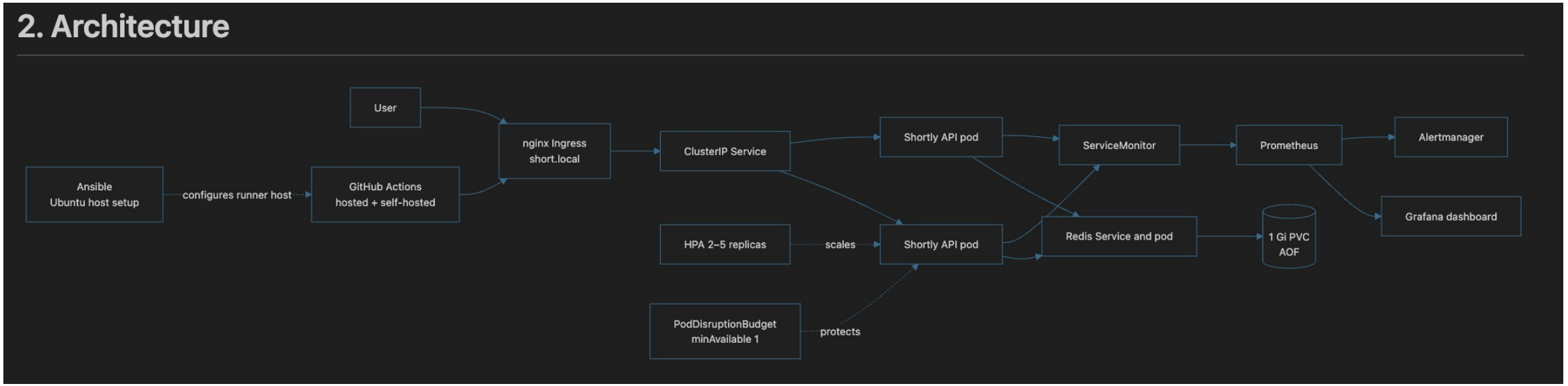
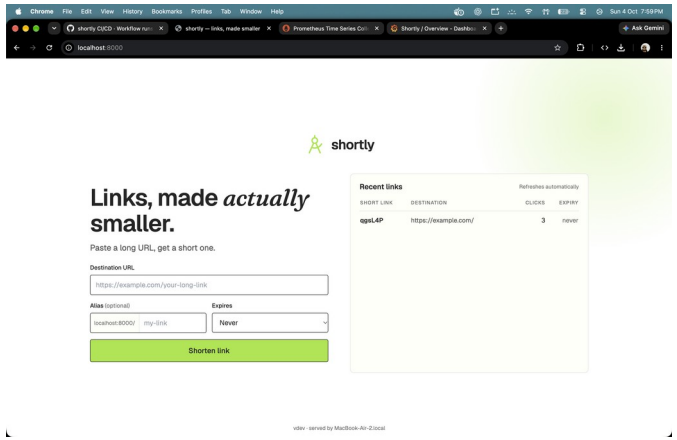
# Shortly turns long URLs into shared, trackable links

Architecture and project team




Captured project architecture diagram



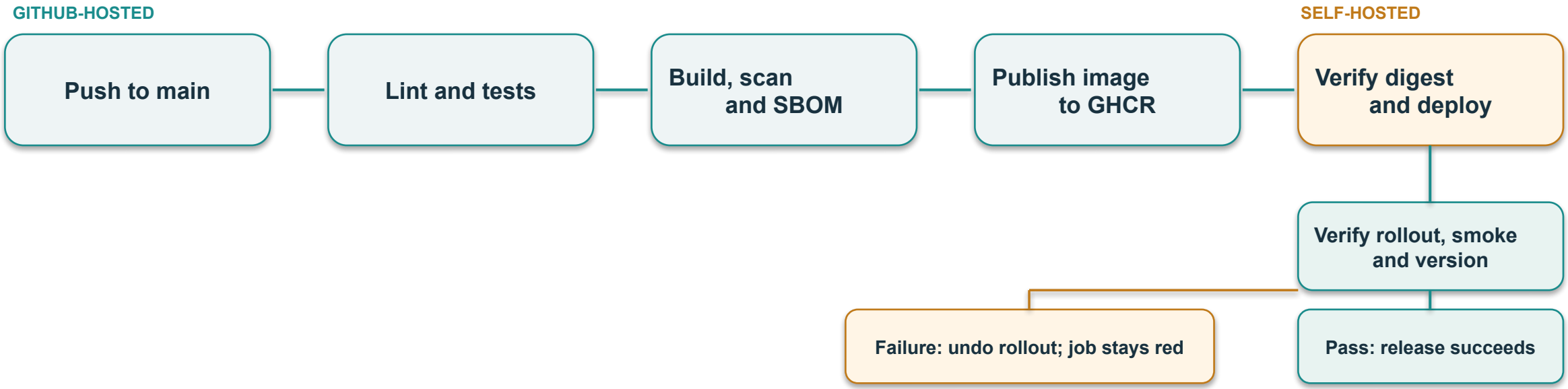

The app creates and lists short links

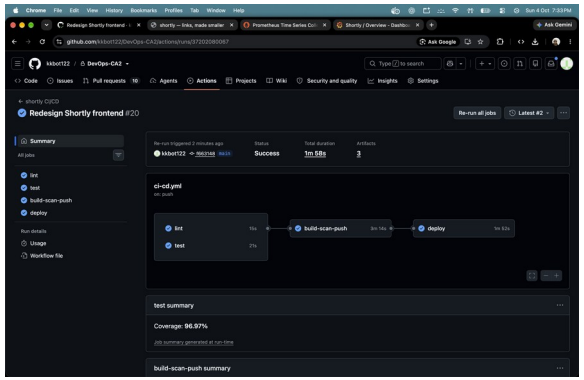
## PROJECT GROUP

| | |
|---|---|
| **Harsh Ledwani** | Roll no. 23070122100 |
| **Shivam Kapure** | Roll no. 23070122113 |
| **Kashyup Gaud** | Roll no. 23070122114 |
| **Kisna Kanti** | Roll no. 23070122116 |

# GitHub Actions checks the image before Minikube deploys it

Hosted build and scan jobs hand the verified image to a guarded local runner



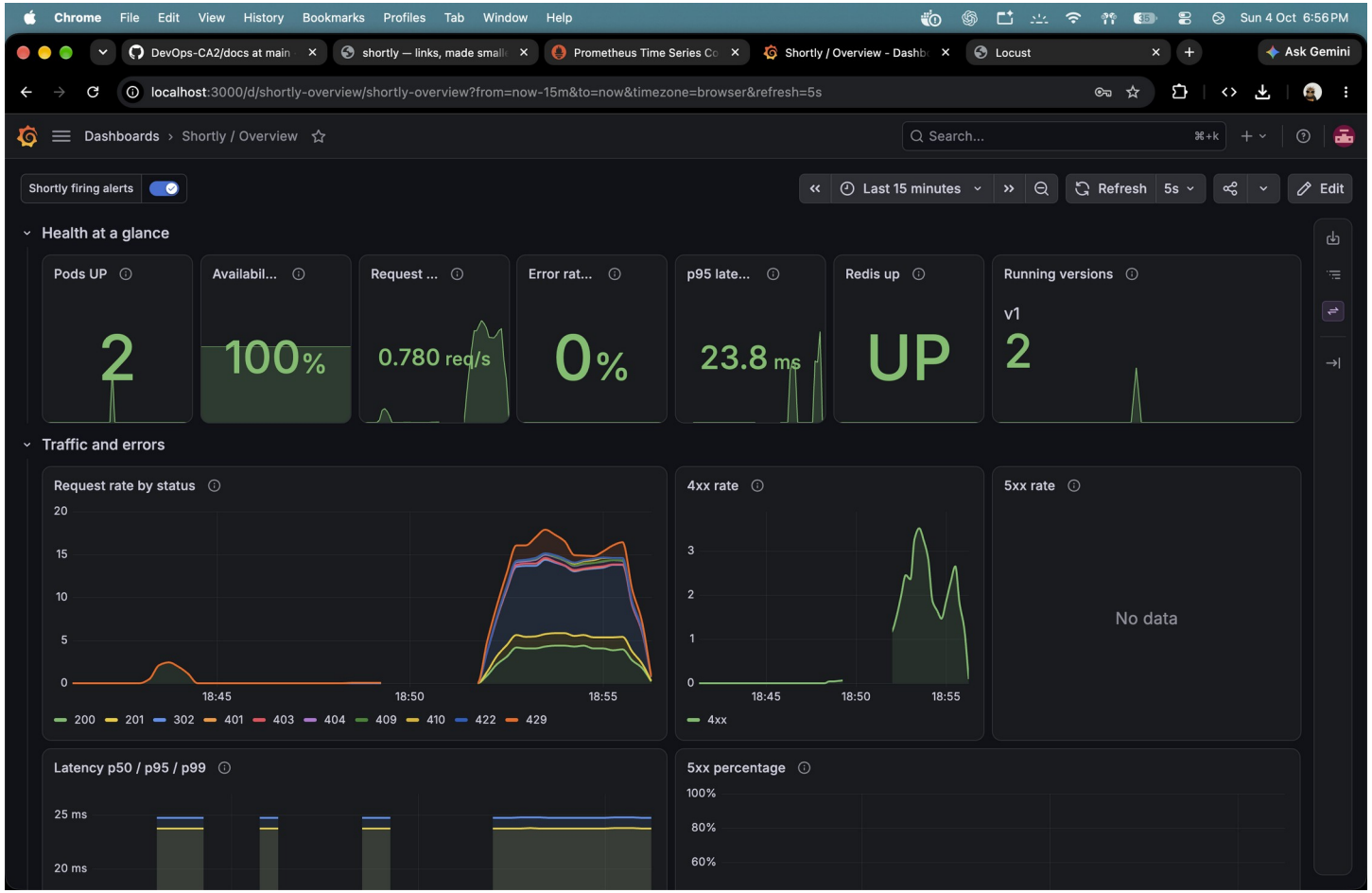

Captured successful workflow run: all jobs passed in 1m 58s

**The deploy job verifies the same image digest that passed the hosted scan.**

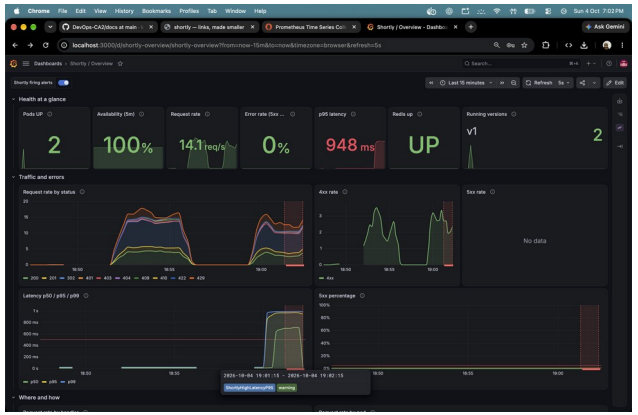
**A verification failure triggers rollback and leaves the workflow visibly failed.**

# Grafana makes traffic, errors, and latency visible

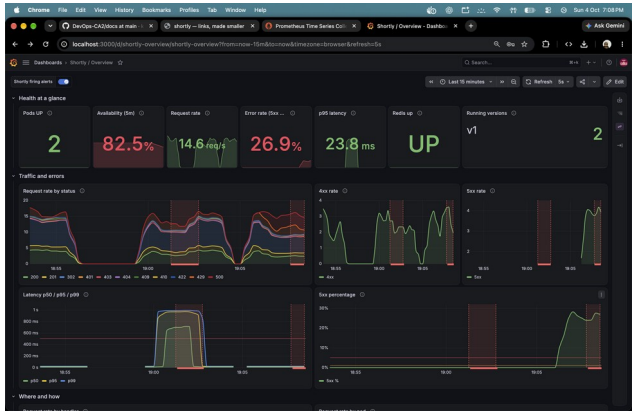
Prometheus scrapes the app; the dashboard turns those metrics into signals




Captured baseline dashboard: 100% availability and 0% 5xx



Latency scenario: p95 rises above the 500 ms threshold



Error scenario: 5xx rate rises during injected failures

**Saved baseline: 3,753 requests · 0 unexpected failures · client p95 8 ms**

# Readiness alone did not catch a broken release

The release stayed Ready while ordinary redirect requests failed

WHAT HAPPENED

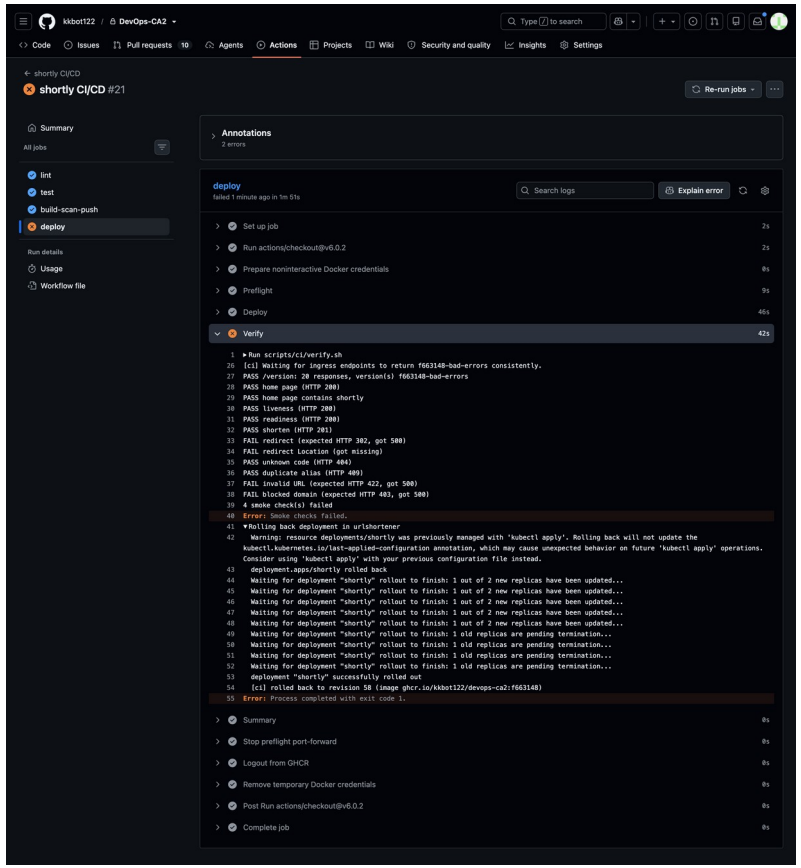
**The bad image passed Kubernetes readiness, but smoke checks found redirect, URL validation, and blocked-domain requests returning 500.**

HOW THE PIPELINE RESPONDED

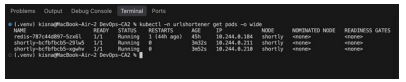
It rolled the Deployment back to revision 58. The workflow stayed failed so the team could see that verification stopped the release.

LESSON

**Health probes check process and dependency health. Smoke tests check user behavior.**




Captured failed release with automatic rollback



Ready API and Redis pods

Ansible runs on a supported Ubuntu 24.04 target. The first apply changed 31 items; the repeat apply changed 0.

# The build taught us to verify releases from the user's side

Results, limits, and the next improvements

01 **Keep readiness and user-facing checks separate.**

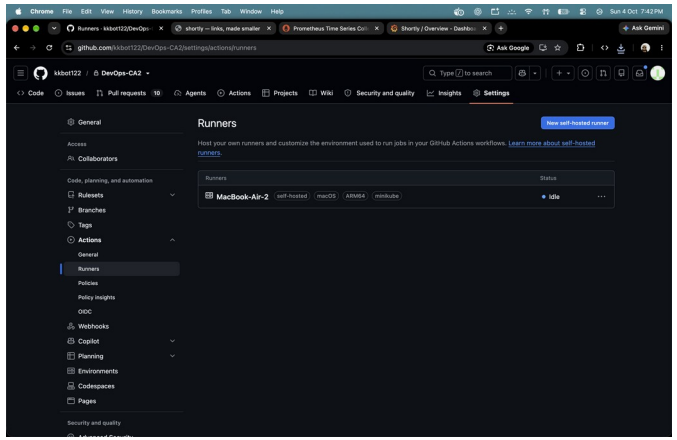
02 Deploy the exact scanned image, then verify requests and version.

03 Use observed resource needs and keep alerts tied to real traffic.

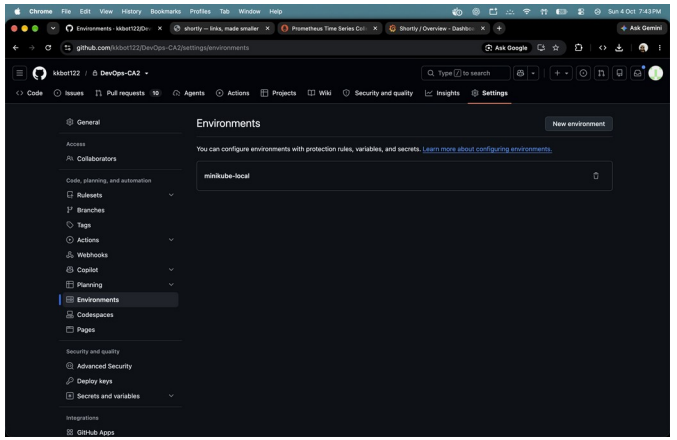
04 Run configuration management on a supported, repeatable host.

**8 of 9 scenarios passed in the saved full run**

The focused abuse rerun later passed. The full suite was not rerun, so the original 8/9 result remains the reported full-run outcome.




Self-hosted deployment runner




GitHub environment configured for local Minikube

**Next: repeat on a multi-node cluster, add a real alert receiver, and isolate the deployment runner.**